

```
# -*- coding: utf-8 -*-
```

```
import logging
import numpy as np
import os
import ast
import pytz
import cartopy.crs as ccrs
import cartopy.feature as cfeature
import matplotlib.pyplot as plt
import matplotlib.colors as mcolors
import sys
# Add the project directory to sys.path
sys.path.insert(0, os.path.abspath('/home/wrf/nons/python-plotting-toolbox'))
from grib_reader import grib_read
from tools import time_tools, reset_folders
from config import CONFIG, DATA_SOURCE, ACCUMULATED_PRECIPITATION,
PRECIP_MULT_FACTOR
```

```
APP_NAME = 'acc_precip'
logger = logging.getLogger(f"carl-wrf-tools.{APP_NAME}")
```

```
APP_CONFIG = CONFIG["plotters"][APP_NAME]
wanted_parameters = ast.literal_eval(APP_CONFIG["parameters"][DATA_SOURCE]
["wantedParameters"])
parameter_names = ast.literal_eval(APP_CONFIG["parameters"][DATA_SOURCE]
["parameterNames"])
locations_file = APP_CONFIG["locationsFile"]
output_time_zone = APP_CONFIG["timeZone"]
start_of_precipitation_day = APP_CONFIG["start_of_precipitation_day"]
accumulation_times = [240] # 10 days in hours
latS, latN, lonW, lonE = APP_CONFIG["latS"], APP_CONFIG["latN"],
APP_CONFIG["lonW"], APP_CONFIG["lonE"]
outdata_path_parent = APP_CONFIG["outdataPath"]
```

```
def main(grib_use_list, output_start_time, output_end_time):
    logger.info('Running 10-day accumulated precipitation calculation...')
    reset_folders.refresh(outdata_path_parent, archive=False)

    cities_dict = {}
    with open(locations_file) as f:
        for line in f.readlines():
            city, lat, lon = map(str.strip, line.split(';'))
            cities_dict[city] = {'lat': float(lat), 'lon': float(lon)}

    acc_dict = {}
    for accumulation_time in accumulation_times:
        outdata_path = os.path.join(outdata_path_parent, str(accumulation_time))
        os.makedirs(outdata_path, exist_ok=True)
        acc_dict[accumulation_time] = {
            'outdata_path': outdata_path,
            'acc_array': None,
            'acc_steps': 0,
            'acc_time_stamps':
time_tools.get_time_stamps_for_acc(accumulation_time, start_of_precipitation_day)
        }
```

```

greenwich_tz = pytz.timezone('Greenwich')
desired_tz = pytz.timezone(output_time_zone) if output_time_zone != 'UTC' else
greenwich_tz

fc_startHour_UTC = output_start_time.astimezone(greenwich_tz)
fc_endHour_UTC = output_end_time.astimezone(greenwich_tz)

grib_valid_times = sorted(time_tools.getForecastValidTime_UTC(grib,
greenwich_tz) for grib in grib_use_list)

data_past = None
for grib_valid_time in grib_valid_times:
    grib_use = next(grib for grib in grib_use_list if
                    time_tools.getForecastValidTime_UTC(grib, greenwich_tz) ==
grib_valid_time)
    base_time_UTC =
greenwich_tz.localize(time_tools.get_forecast_basetime_UTC(grib_use))
    validTime_UTC = time_tools.getForecastValidTime_UTC(grib_use, greenwich_tz)

    if validTime_UTC == base_time_UTC:
        logger.warning('Skipping first forecast time step!')
        continue

    lats, lons, data = grib_read.main(grib_use, parameter_names,
wanted_parameters, grib_use_list)

    total_precipitation = (data[parameter_names[0]] - data_past[
        parameter_names[0]]) if ACCUMULATED_PRECIPITATION and data_past else
data[parameter_names[0]]
    total_precipitation[total_precipitation < 0] = 0
    total_precipitation *= PRECIP_MULT_FACTOR

    for accumulation_time in acc_dict:
        if acc_dict[accumulation_time]['acc_array'] is None:
            acc_dict[accumulation_time]['acc_array'] = np.zeros_like(lats)

            acc_dict[accumulation_time]['acc_array'] += total_precipitation
            acc_dict[accumulation_time]['acc_steps'] += 1

            if acc_dict[accumulation_time]['acc_steps'] >= 80: # 80 steps for 10
days assuming 3-hourly data
                logger.info(f'Creating plot for {accumulation_time}-hour
accumulation')
                create_plot(lats, lons, acc_dict[accumulation_time]['acc_array'],
validTime_UTC,
                           acc_dict[accumulation_time]['outdata_path'], 240,
cities_dict, latS, latN, lonW, lonE)
                acc_dict[accumulation_time]['acc_array'] = np.zeros_like(lats)
                acc_dict[accumulation_time]['acc_steps'] = 0

    data_past = data

def create_plot(lats, lons, acc_array, validTime_UTC, outdataPath, acc_steps,
cities_dict, latS, latN, lonW, lonE):

    projLat = np.mean([latS, latN])
    projLon = np.mean([lonW, lonE])
    min_latitude = latS-1

```

```

max_latitude = latN+1

max_precip = 10
levs = [1, 2.5, 5, 7.5, 10, 15, 20, 30, 40, 50, 70, 100, 150, 200, 250, 300,
400, 500, 600, 750]
clevs = clevs = [0, 1, 2.5, 5, 7.5, 10, 15, 20, 30, 40, 50, 70, 100, 150, 200,
250, 300, 400, 500, 600, 750]#np.linspace(0.1, max_precip, 21) #[0, 0.2, 0.4, 0.6,
0.8, 1., 1.2, 1.4, 1.6, 1.8, 2.0, 2.2, 2.4, 2.6, 2.8, 3., 3.2, 3.4, 3.6, 3.8, 4.,
4.2, 4.4, 4.6, 4.8, 5., 5.2, 5.4, 5.6, 5.8, 6.]

fig = plt.gcf()
ax = fig.add_subplot(1,1,1, projection =
ccrs.Mercator(central_longitude=projLon, min_latitude = min_latitude, max_latitude
= max_latitude))
#ax.coastlines(resolution='10m', linewidth=0.3)

ax.add_feature(cfeature.RIVERS.with_scale('10m'), linewidth=0.4)
ax.add_feature(cfeature.LAKES.with_scale('10m'), linewidth=0.4)
ax.add_feature(cfeature.BORDERS.with_scale('10m'), linewidth=0.6)
ax.add_feature(cfeature.STATES.with_scale('10m'), linewidth=0.6)

ax.set_extent([lonW, lonE, latS, latN], crs=ccrs.PlateCarree())
ax.set_extent([lonW, lonE, latS, latN], crs = ccrs.PlateCarree())
cmap_data = [(1.0, 1.0, 1.0),
(0.3137255012989044, 0.8156862854957581, 0.8156862854957581),
(0.0, 1.0, 1.0),
(0.0, 0.8784313797950745, 0.501960813999176),
(0.0, 0.7529411911964417, 0.0),
(0.501960813999176, 0.8784313797950745, 0.0),
(1.0, 1.0, 0.0),
(1.0, 0.6274510025978088, 0.0),
(1.0, 0.0, 0.0),
(1.0, 0.125490203499794, 0.501960813999176),
(0.9411764740943909, 0.250980406999588, 1.0),
(0.501960813999176, 0.125490203499794, 1.0),
(0.250980406999588, 0.250980406999588, 1.0),
(0.125490203499794, 0.125490203499794, 0.501960813999176),
(0.125490203499794, 0.125490203499794, 0.125490203499794),
(0.501960813999176, 0.501960813999176, 0.501960813999176),
(0.8784313797950745, 0.8784313797950745, 0.8784313797950745),
(0.93333333373069763, 0.8313725590705872, 0.7372549176216125),
(0.8549019694328308, 0.6509804129600525, 0.47058823704719543),
(0.6274510025978088, 0.42352941632270813, 0.23529411852359772),
(0.4000000059604645, 0.20000000298023224, 0.0)]

cmap = mcolors.ListedColormap(cmap_data, 'precipitation')
norm = mcolors.BoundaryNorm(clevs, cmap.N)
cs = ax.contourf(lons, lats, acc_array, clevs, transform=ccrs.PlateCarree(),
cmap=cmap, norm=norm)
fig.colorbar(cs, ticks = clevs)
cl = ax.contour(lons, lats, acc_array, levs, colors='k',
transform=ccrs.PlateCarree(), linewidths = 0.1)
plt.clabel(cl, inline=1, fmt='%1.f', fontsize=8)

lat_cities = []
lon_cities = []
labels = []
for key in cities_dict:
    labels.append(key)

```

```

lat_cities.append(cities_dict[key]['lat'])
lon_cities.append(cities_dict[key]['lon'])

ax.plot(lon_cities, lat_cities, 'ko', markersize=4,
transform=ccrs.PlateCarree())
for label, xpt, ypt in zip(labels, lon_cities, lat_cities):
    plt.text(xpt, ypt, label, transform=ccrs.PlateCarree())

titleString = "Precipitation accumulated over {} hours, mm \n Valid:
{}".format(str(acc_steps), validTime.UTC.strftime('%Y-%m-%d %H UTC')) +
validUTC_dt.strftime('%Y-%m-%d %H:00') + " UTC"
plt.title(titleString)
figure = plt.gcf()
figure.set_size_inches(16, 9)
save = outdataPath + validTime.UTC.strftime('%Y-%m-%dT%H%M') +
'_acc{0}H.png'.format(acc_steps)
plt.savefig(save, dpi=200)
plt.clf()

if __name__ == '__main__':
    grib_use_list = [...] # Define or load the list of GRIB files
    output_start_time = '2025-03-15 00:00' # Adjust to your needs
    output_end_time = '2025-03-15 12:00' # Adjust accordingly

    main(grib_use_list, output_start_time, output_end_time)

```